Minería de Datos en Python

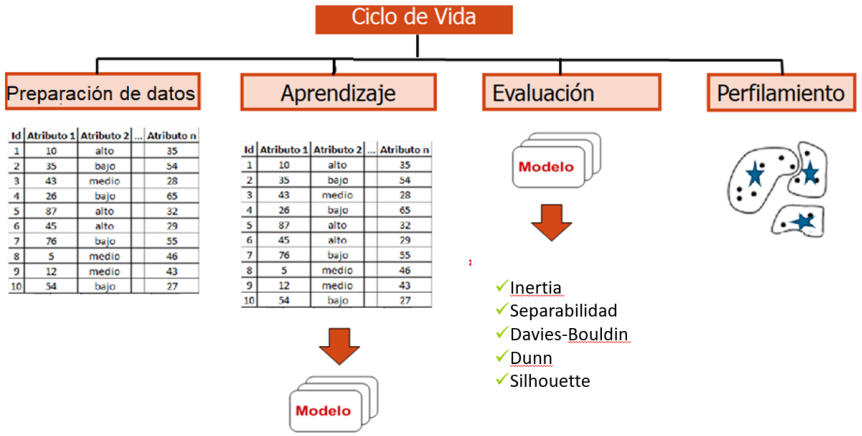
Magister: Efraín Alberto Oviedo
efrain.oviedo@upb.edu.co

UNIVERSIDAD PONTIFICIA BOLIVARIANA
FACULTAD DE INGENIERÍA EN TIC
2025

Universidad Pontificia Bolivariana

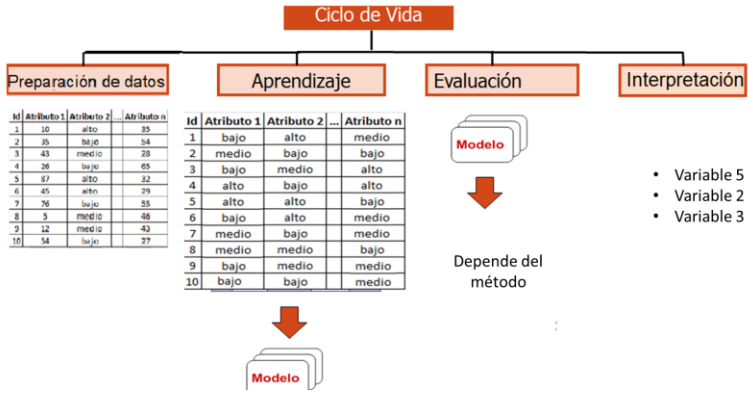
Minería Descriptiva

Clustering



Asociación

Selección de Factores



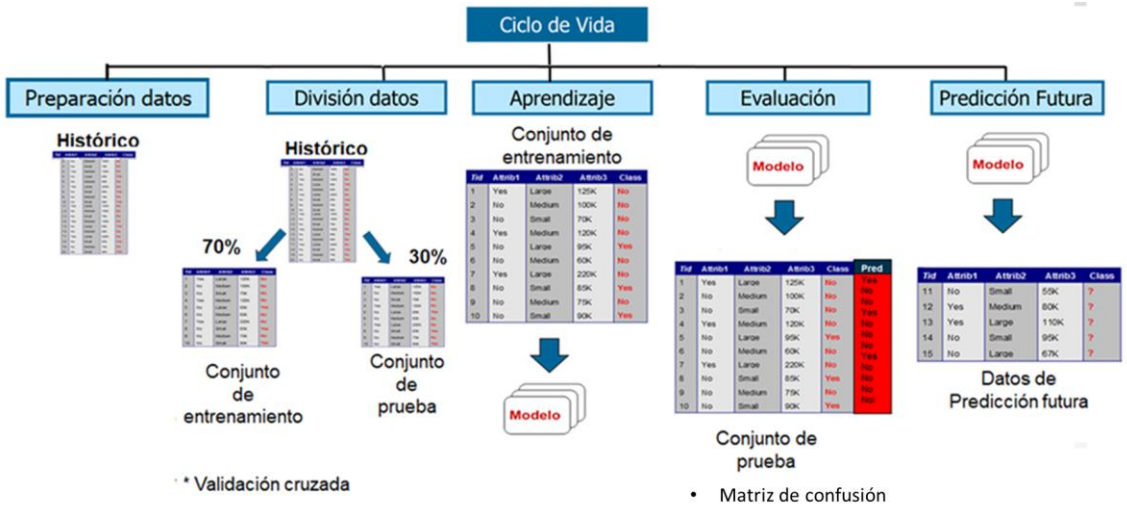
Análisis	Medidas de Evaluación	Algoritmo	Parámetros de Configuración	Código Python
Clustering: descubrimiento de similitudes	Inertia, separabilidad, silueta, Indice Dun, Indice DB	Kmeans	Cantidad de clusters, cantidad de iteraciones	`from sklearn.cluster import KMeans` `model = KMeans(n_clusters=3, max_iter=100)`

Reglas de Asociación: descubrimiento de co-ocurrencias (patrones repetidos)	Soporte y confianza	Apriori	Soporte mínimo y confianza mínima	`from apyori import apriori`
				`reglas=apriori(data.values, min_support=0.1, min_confidence=0.6)`
				`reglas=list(reglas)`

Selección de Factores: descubrimiento de variables relevantes	Depende del método	Correlation
		Arbol de decisión
		Regresión lineal/logística
			Correlation: NA
			Arbol: criterio, cantidad de registros en las hojas, profundidad
			Regresion: si se desea normalizar, si es multiclase
				`data.corr()`
				`from sklearn.tree import DecisionTreeClassifier`
				`model = DecisionTreeClassifier(criterion='gini', min_samples_leaf=2, max_depth=10)`
				`from sklearn.linear_model import LogisticRegression`
				`model = LogisticRegression(solver='liblinear', multi_class='auto')`

Minería Predictiva

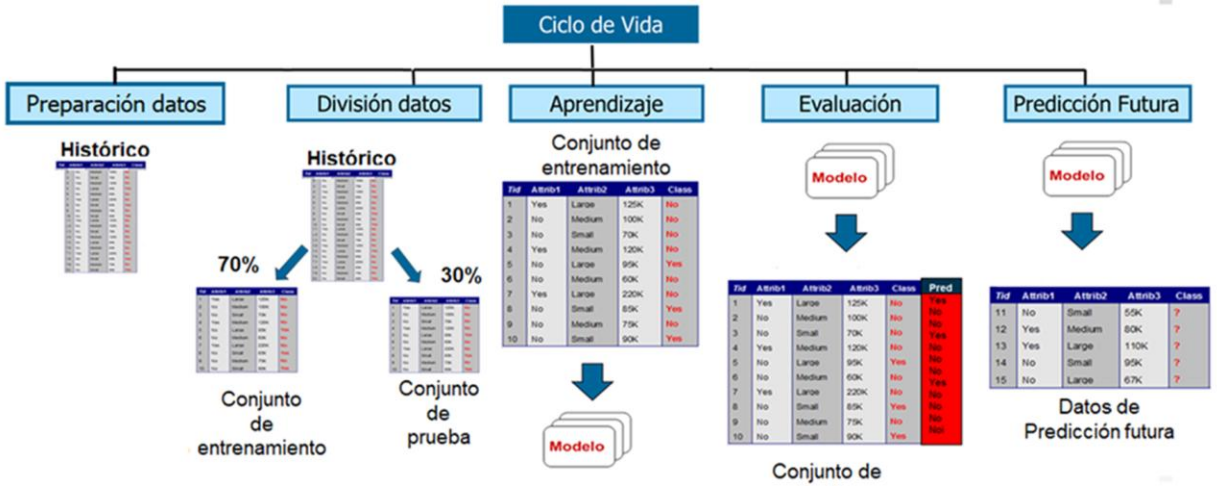
Clasificación



* Validación cruzada

- Matriz de confusión
- Precision
- Recall
- F1
- Area ROC

Regresión



* Validación cruzada

- MSE
- RMSE
- MAE
- MRAE

Minería Predictiva

Análisis	Medidas de Evaluación	Algoritmo	Parámetros de Configuración	Código Python
Clasificación: predicción de una variable objetivo categórica	Matriz de confusión, presicion, recall, exactitud, f1, area ROC	Arboles de Clasificación	Indice de información (entropy, gini), mínima cantidad de datos en las hojas, máxima profundidad	`from sklearn.tree import DecisionTreeClassifier` `model = DecisionTreeClassifier(criterion='gini', min_samples_leaf=2, max_depth=10)`
		KNN	Cantidad de vecinos, medida de distancia	`from sklearn.neighbors import KNeighborsClassifier` `modelKnn = KNeighborsClassifier(n_neighbors=1, metric='euclidean')`

		Redes Neuronales	Cantidad de capas y neuronas, función de activación, tasa de aprendizaje, valor inicial de la tasa de aprendizaje, momentum, cantidad de iteraciones	`from sklearn.neural_network import MLPClassifier`
				`modelRN = MLPClassifier(activation="logistic",hidden_layer_sizes=(4), learning_rate='constant',`
				`learning_rate_init=0.2, momentum= 0.3, max_iter=500)`

		SVM	kernel	`from sklearn.svm import SVC` `modelSVM = SVC(kernel='linear') #'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'`
		Otros: Regresión logística, bayesiano		

Regresión: Predicción de una variable objetivo numérica	(MSE) – Error cuadrático medio
	(RMSE) -Raíz del error cuadrático medio
	(MAE) -Error medio absoluto
	(R^2) – R cuadrado
	(MRAE) – Error relativo absoluto
		Arboles de Regresión	Indice de información (mse, mae), mínima cantidad de datos en las hojas, máxima profundidad	`from sklearn.tree import DecisionTreeRegressor`
				`model_Tree = DecisionTreeRegressor(criterion='mse', min_samples_leaf=2,`
				`max_depth=None)`

		KNN para Regresión	Cantidad de vecinos, medida de distancia	`from sklearn.neighbors import KNeighborsRegressor` `model_Knn = KNeighborsRegressor(n_neighbors=1, metric='euclidean')`

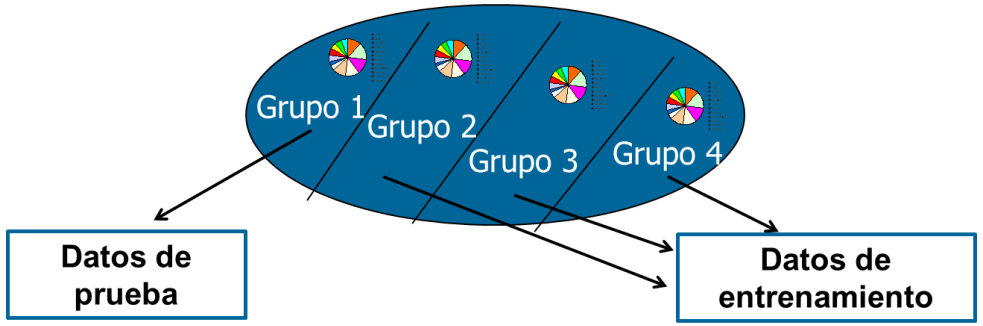
		Redes Neuronales	Cantidad de capas y neuronas, función de activación, tasa de aprendizaje, valor inicial de la tasa de aprendizaje, momentum, cantidad de iteraciones	`from sklearn.neural_network import MLPRegressor`
				`model_RN = MLPRegressor(activation="logistic",hidden_layer_sizes=(4), learning_rate='constant',`
				`learning_rate_init=0.2, momentum= 0.3, max_iter=500)`

		SVM para Regresión	kernel	`from sklearn.svm import SVR` `model_SVM = SVR(kernel='linear') #'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'`
		Otros: series de tiempo, regresión lineal		

Validación Cruzada

```
from sklearn import tree
from sklearn.model_selection import cross_validate
```

```
model = tree.DecisionTreeClassifier(min_samples_leaf=2000, max_depth=10)
scores = cross_validate(model, X, Y, cv=10, scoring=('f1', 'accuracy','precision',
                        'recall','roc_auc'), return_train_score=False)
```



Universidad Pontificia Bolivariana

Balanceo

```
#sm = SMOTE() para variables predictoras numéricas
from imblearn.over_sampling import SMOTE

#sm = SMOTENC() para variables predictoras categóricas
from imblearn.over_sampling import SMOTENC

sm = SMOTENC(categorical_features=[1,2,3,4,5,8,9,10,15]) #se indican las var categóricas
X_bal, Y_bal = sm.fit_resample(X_train,Y_train)

# Creamos un dataframe con los resultados
data_bal  = pd.DataFrame(columns=X_train.columns.values, data=X_bal)
data_bal['Objetivo']=Y_bal
```

Predicción de inversión en una tienda de videojuegos

Edad

20

14 52

Videojuego

'Mass Effect'

Plataforma

'Play Station'

Sexo

Hombre

Consumidor_habitual

True

Predicciones

	Edad	videojuego	Plataforma	Sexo	Consumidor_habitual	Tree	Knn	Prediccion_NN
0	20	'Mass Effect'	'Play Station'	Hombre	True	410	500	347.0949

Despliegue Github- Streamlit

1. Crea una cuenta en GitHub

- Ve a https://github.com
- Haz clic en Sign up
- Sigue los pasos para crear tu cuenta (usuario, correo, contraseña)

requirements.txt
pandas
numpy
matplotlib
scikit-learn

2. Crea un nuevo repositorio en GitHub

- Inicia sesión en GitHub
- Haz clic en el botón "+" en la esquina superior derecha > New repository
- Ponle un nombre, por ejemplo: mi-app-streamlit
- Marca la opción "Add a README file"
- Haz clic en Create repository

3. Sube tu proyecto a GitHub

- En tu repositorio, haz clic en Add file > Upload files
- Arrastra tus archivos (app.py, requirements.txt, modelo.pkl.). También puedes subir y trabajar colaborativamente los notebooks de creación de los modelos, pero ten cuidado si son datos confidenciales.
- Haz clic en Commit changes

Universidad Pontificia Bolivariana

Despliegue Github- Streamlit

4. Conecta tu proyecto con Streamlit Community Cloud

- Ve a https://streamlit.io/cloud
- Inicia sesión con tu cuenta de GitHub. Ve a Free/Login with Github
- Haz clic en "Create app"/Deploy a public app from Github
- Selecciona:
- El repositorio (ej. tu-usuario/mi-app-streamlit)
- La rama (generalmente main)
- El archivo principal (app.py)
- Haz clic en Deploy

Despliegue Github- Streamlit

Para hacer actualizaciones

- Ve a tu repositorio en GitHub.
- Abre el archivo requirements.txt.
- Haz clic en el ícono de lápiz ✏️ (Editar).
- Baja hasta el final y haz clic en Commit changes.
- Ve a tu app en Streamlit Community Cloud.
- Usa el botón de menú → "Clear cache and restart"

Ejercicio Github - Streamlit

- Y si hacemos el despliegue con varios modelos?
- Que tal promediar la predicción de los dos modelos con menor error?
- Y si entregamos el mínimo y el máximo de la predicción?

Predicción de inversión en una tienda de videojuegos

Edad

25

Videojuego

'Fifa'

Plataforma

'Xbox'

Sexo

Mujer

Consumidor_habitual

True

Predicciones

	Edad	videojuego	Plataforma	Sexo	Consumi	Pred_TREE	Pred_RF	Pred_KNN	Pred_NN
0	25	'Fifa'	'Xbox'	Mujer	True	400	414.0654	900	409.6233